[RFC]

Migración de Arquitectura

De Monolito Tradicional a Monolito Modular

Autoras : Gabriela Navarro-20000127 · Sandra Soria-20002619

Proyecto : Spring PetClinic · Arquitectura de Software · Marzo 2026

1. Executive Summary

En este documento se propone una reestructuración arquitectónica del sistema Spring PetClinic, partiendo de su arquitectura actual un monolito tradicional hacia un enfoque de monolito modular.

Actualmente, la aplicación tiene todos sus dominios de negocio conviviendo en el mismo proyecto sin fronteras claras entre ellos. Lo que proponemos es organizarla en módulos separados por dominio (Owners, Pets, Visits y Vets), pero sin dejar de ser una sola aplicación desplegable.

Con este cambio buscamos:

- Mejorar la mantenibilidad del sistema.
- Reducir el acoplamiento entre componentes.
- Preparar el sistema para una posible migración futura a microservicios.

2. Motivation

Spring PetClinic es una aplicación monolítica construida con Spring Boot. Esta arquitectura es simple y fácil de desplegar, pero puede generar dificultades cuando el sistema crece o cuando varios desarrolladores trabajan en el simultáneamente.

Problema	Impacto
Sin separación clara entre dominios	Dificulta entender el sistema
Alto acoplamiento entre componentes	Un cambio puede romper cosas inesperadas
Dificultad para escalar el equipo	Varios devs tocando el mismo código
Mantenimiento cada vez más costoso	El tiempo para hacer cambios aumenta
Deuda técnica acumulada	Cada feature nuevo suma complejidad

Las practicas modernas de arquitectura de software sugieren establecer límites claros entre los dominios antes de considerar una migración hacia microservicios. El patrón de monolito modular permite lograr esta separación manteniendo la simplicidad de una sola aplicación.

Además, este enfoque se alinea con los principios de Domain-Driven Design (DDD), que organizan el sistema alrededor de los dominios del negocio.

---

3. Proposed Implementation

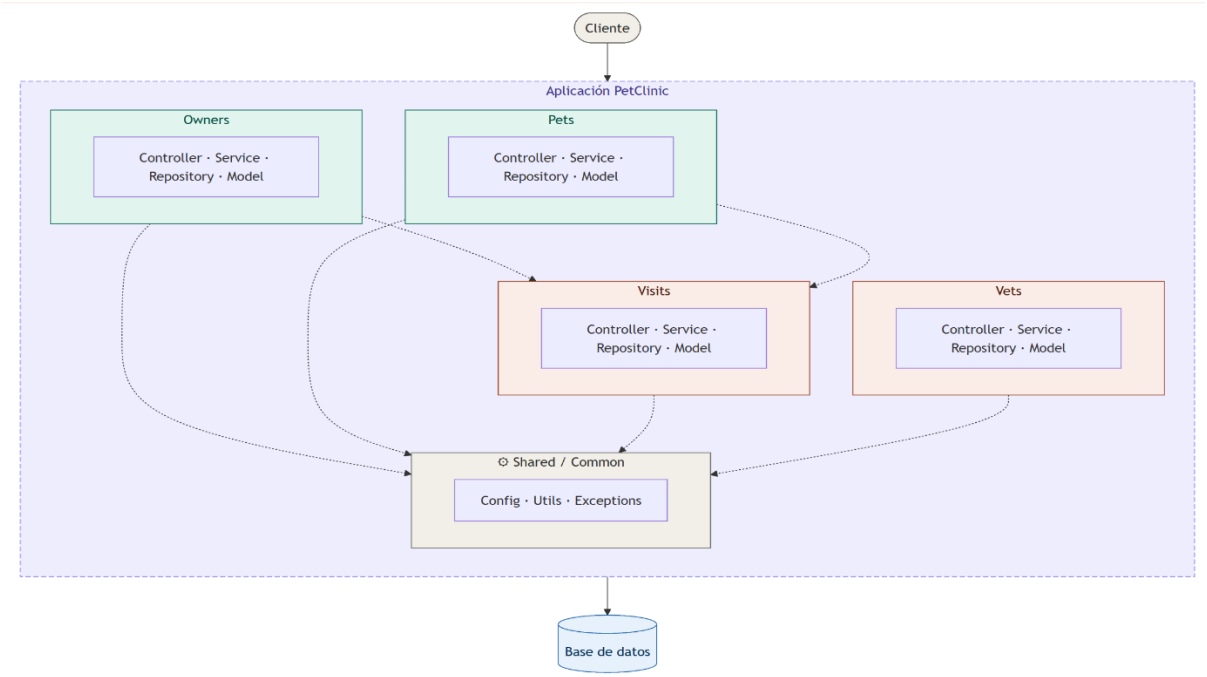
La idea es reorganizar internamente el sistema dividiéndolo en módulos por dominio de negocio. La aplicación seguirá siendo una sola unidad desplegable en Spring Boot, pero su estructura interna será mucho más clara.

Módulos propuestos

```
src/main/java/petclinic/
├── owners/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   └── model/
│
├── pets/ · visits/ · vets/ (misma estructura)
│
└── shared/
    ├── config/
    ├── exceptions/
    └── util/
```

Cada módulo contendrá sus propios controllers, services, repositorios y modelos de dominio. La comunicación entre módulos se realizará a través de interfaces bien definidas, con el objetivo de reducir el acoplamiento y evitar dependencias innecesarias entre componentes.

Diagrama de arquitectura



4. Metrics & Dashboards

Para evaluar si esta propuesta genera mejoras reales, se definirán métricas específicas que puedan medirse antes y después de la implementación. Estas métricas serán analizadas mediante el pipeline de gobernanza configurado en GitHub Actions.

Métricas de código

Herramienta: SonarQube

- Complejidad ciclomatica por modulo
- Cobertura de pruebas (Code Coverage)
- Numero de dependencias entre módulos
- Tamaño de cada módulo (líneas de código, clases)

Métricas DORA

Las métricas DORA son un estándar reconocido para medir la salud del proceso de desarrollo:

Métrica	Descripción
Deployment Frequency	¿Con que frecuencia desplegamos?
Lead Time for Changes	¿Cuánto tarda un cambio en llegar a producción?
Change Failure Rate	¿Qué % de despliegues genera incidentes?
MTTR	¿Cuánto tardamos en recuperarnos de un fallo?

Experiencia del desarrollador

- Tiempo que le toma a alguien nuevo entender un dominio del sistema.
- Tiempo de onboarding general en el proyecto.
- Facilidad percibida para hacer cambios en un módulo sin romper otros.

5. Drawbacks

Como cualquier propuesta de arquitectura, esta decisión también implica ciertos costos y riesgos. Los más importantes son:

Riesgo principal:

Si el equipo no mantiene disciplina arquitectónica, el sistema podría terminar siendo igual de acoplado que al principio, pero con más capas de complejidad. Esto se conoce como el antipatrón Big Ball of Mud, donde el sistema pierde su estructura y se vuelve difícil de mantener.

Otros posibles inconvenientes incluyen:

- La refactorización inicial requiere tiempo e implica modificar partes del código existente.
- El equipo debe adoptar una disciplina arquitectónica real para respetar los límites entre módulos.

- Es un esfuerzo que no introduce funcionalidades nuevas directamente, aunque sí mejora la mantenibilidad y la organización del sistema.

---

6. Alternatives

Antes de decidir por el Monolito Modular, evaluamos otras dos opciones:

Opción A — Migración directa a microservicios

Ventajas

- Alta escalabilidad.
- Despliegue independiente por servicio.
- Mayor aislamiento entre componentes.

Desventajas

- Mucha mayor complejidad operativa.
- Necesidad de infraestructura adicional (service mesh, orquestación, etc.).
- Costo de mantenimiento considerablemente más alto.

Considerando la escala actual del sistema y del equipo, esta opción no se considera necesaria por ahora.

Opcion B — Mantener el monolito actual

Ventajas

- No requiere refactorización.
- Es la opción más simple a corto plazo.

Desventajas

- La deuda técnica seguirá creciendo.
- El sistema se vuelve cada vez más difícil de escalar.
- El riesgo de acoplamiento aumenta con cada feature nuevo.

¿Por qué el Monolito Modular?

El enfoque de Monolito Modular ofrece un equilibrio adecuado entre simplicidad y organización. Permite mejorar la estructura interna del sistema sin asumir todavía la complejidad operativa de una arquitectura de microservicios.

---

7. Potential Impact and Dependencies

Impacto en el equipo de desarrollo

Los desarrolladores deberán trabajar respetando los límites definidos entre los módulos. A cambio, diferentes equipos podrán trabajar en distintos dominios del sistema con un menor riesgo de interferir en el trabajo de otros.

Impacto en la infraestructura

Uno de los principales beneficios de esta propuesta es que la aplicación continúa siendo una única unidad desplegable, por lo que no se requieren cambios significativos en la infraestructura.

Impacto en la Seguridad

Una separación más clara entre módulos que puede ayudar a reducir el riesgo de accesos no autorizados a datos o funcionalidades pertenecientes a otros dominios. Sin embargo, si las interfaces entre módulos no se diseñan adecuadamente, podrían introducirse vulnerabilidades internas que deberán gestionarse con cuidado.

---

8. Unresolved Questions

Hay algunas preguntas que dejamos abiertas para futuras decisiones arquitectónicas:

- ¿Alguno de los módulos debería eventualmente convertirse en un microservicio independiente?
- ¿Qué nivel de aislamiento es el adecuado entre módulos? ¿Bases de datos separadas o únicamente comunicación mediante interfaces?
- ¿Qué estrategia de pruebas automatizadas tiene más sentido aplicar por módulo?

Estas son decisiones que tiene sentido tomar una vez que la arquitectura modular esté implementada y podamos observar cómo se comporta el sistema en la práctica.

9. Costos

Refactorización inicial

La reorganización del proyecto para dividir el código en módulos requiere dedicar tiempo de desarrollo exclusivamente a esta tarea, sin agregar nuevas funcionalidades al sistema.

Tiempo estimado

Escenario	Duración estimada
Equipo pequeño (1-2 personas)	~4 semanas
Equipo mediano (3-4 personas)	~2 semanas

La duración puede variar dependiendo de cuántas dependencias cruzadas existan actualmente entre los diferentes dominios del sistema.

Aprendizaje

La implementación de una arquitectura modular requiere que el equipo comprenda y respete los límites entre los diferentes dominios del sistema. Aunque esto implica un pequeño esfuerzo de aprendizaje inicial, con el tiempo permite mejorar la mantenibilidad del software y reducir la deuda técnica acumulada.

10. Conclusión

Migrar a una arquitectura de Monolito Modular representa una mejora concreta y alcanzable sobre el estado actual de Spring PetClinic. No es la solución más ambiciosa posible, pero sí la más adecuada considerando la situación actual del sistema. Con esta propuesta se busca:

- Mejorar la organización del código sin reemplazar completamente lo que ya existe.
- Reducir el acoplamiento entre los diferentes dominios del negocio.
- Facilitar el trabajo en equipo, permitiendo que cada desarrollador o equipo sea responsable de su propio módulo.
- Preparar el sistema para una posible evolución futura hacia microservicios, en caso de que el crecimiento del proyecto lo requiera.

Todo esto se logra sin introducir la complejidad operativa que implicaría migrar directamente a una arquitectura distribuida.